

Hive Inspect

Forward Deployed Engineer

Build a template importer

Inspectors moving to new software arrive carrying templates they have spent years tuning. Getting those across intact is the difference between a customer who switches and one who does not. You are going to build that workflow: import a template, let the inspector work with it, and keep their changes.

Deadline: By 21 September 2026. If you need an extension, let us know as soon as possible and we will accommodate it.

Expected effort: Two focused days, hackathon style, including product exploration, building, deployment, and your walkthrough. AI coding tools are highly encouraged.

Step one is to use the real thing

Before you start building, sign up for a free Hive Inspect trial. Run a sample inspection through and publish a report, then try the template-import workflow. Documentation is at docs.hiveinspect.com.

Product	Access	Expectation
Hive Inspect	Five free reports, no credit card	Required
Binsr	Five free inspections, no card	Optional, highly encouraged
Spectora	Two free reports, no credit card	Required

If you explore Binsr, check whether Hive and Binsr approach template import differently. What does each make easier, and what could Hive learn from Binsr? If you only explore Hive, explain that choice. The comparison is encouraged, not required.

Get your input file

We are not supplying a template. Sign up for Spectora, load an available template such as InterNACHI Residential, and export it using **Export to spreadsheet → Export HTML Text**. Use the resulting spreadsheet file as your input, rather than the plain-text export.

Commit that export to your repo, so we can test against the same file. Note which template you used and where it came from. Use sample material you can share, without real customer information.

The project

Take a Spectora HTML-text template export, bring it into your own system, and let the user work with it from there.

The customer is an inspection company coming off Spectora with a template they have tuned for four years. They will not retype it. **Preserving that work matters more than originality.**

The working baseline

1. **Import.** Let the user upload a Spectora HTML-text export. Preserve the template's text, hierarchy, and ordering. Make skipped or unsupported content visible; do not quietly drop or rewrite it.
2. **Edit.** Let the user change section names, item names, and comment text after import, then save those changes. Decide how much further the editor should go.
3. **Copy.** Let the user duplicate a template and edit the copy independently. Changes to the copy must leave the original unchanged.
4. **Store.** Save templates, edits, and copies in a real backend. They must still be retrievable after closing and reopening the app.
5. **Model it.** Import into a schema you designed: templates, sections, items, comments, or another structure you can explain. The template must be structured and editable. HTML inside individual comment fields is fine; a whole template stored as one opaque HTML blob is not.

Explain how you handle formatting, links, and other rich content in the export, including any limits. Distinguish information missing from the export from information your importer does not support.

We may try another export in the same HTML-text format. Your importer should work beyond the exact template you committed. Show how you checked preservation, saved edits, and independent copies, and how the app handles at least one failure case.

Choose where to go further

Get the baseline working first. Then choose one improvement that matters to this customer: make the import easier to trust, make the editor easier for a non-technical inspector, or handle a difficult case well. Explain the customer problem and why you spent time on it.

We do not expect a complete product in two days. Leave further features out deliberately and document why. We care most about faithful import, a usable workflow, and sound decisions about where to spend your time.

Out of scope: Writing actual inspection reports, scheduling, payments, and homeowner-facing reports or portals.

What to build

- **A web app.** The person using this workflow is at a desk; a mobile app is out of scope.
- **A real backend with real persistence.** Browser storage alone is not sufficient. Supabase is encouraged; another real database is fine.
- **A working public URL, deployed on Vercel.** We will review the live app. If your stack cannot reasonably run on Vercel, host it elsewhere and tell us where. If you add a login, include access instructions.

Use the stack you are fastest in. Existing libraries, starters, and open-source projects are welcome. Credit what you build on and explain your contribution. **Seed the live app with a template you have already imported**, so it opens with something we can explore.

Use AI tools

To build this: Please do. Claude, Cursor, Copilot, agents, or whatever you actually work with. Use existing products as references and let your coding tools help you understand, build, and refine the solution. You are responsible for understanding and checking what you ship.

Inside the product itself: Your call. Explain the decision in your walkthrough. If you use a model for import mapping, show what happens when it returns malformed output, invents sections, or drops content. Validation and honest failures matter. These expectations also apply to importers that do not use a model.

If you create reusable prompts, skills, agent setups, or scripts along the way, include them in the repo. We are interested in how you work.

What to send us

1. **A repo.** Include meaningful development history, the Spector export you used, and a short README with setup, database initialization, and environment-variable instructions. Keep credentials out of the repo. If the repo is private, provide reviewer access.
2. **A live URL.** Open on an imported template. Include login instructions if needed.
3. **A walkthrough video.** Follow the outline below.
4. **NOTES.md.** Explain what you cut and why, supported input and known limitations, how you checked your work, and approximate time spent. Credit existing code or starters you built on.

The walkthrough video

Eight to ten minutes, twelve at the most. Show your screen and explain in your own voice. Turn your camera on for the introduction; it is optional for the rest. Rough is fine, and an outline or notes are welcome.

Record however is easiest. OBS or a Zoom meeting with yourself works. Upload to **YouTube as unlisted**, or use another video link we can open without requesting access. Loom is fine if your plan supports the full length.

1. **You.** Briefly introduce yourself and what you have worked on.
2. **What you built.** Import the Spector file on camera. Show an edit being saved and a copy being changed independently of the original.
3. **The repo.** Explain the layout, stack, any existing code you started from, and how you used AI coding tools.
4. **The data model.** Explain the database structure, the import mapping, and how you checked that the customer's content survived.
5. **Your decisions.** Explain what you prioritised and what you left out. If you added an improvement, connect it to the customer's problem. If you explored Binsr, compare it with Hive and explain what informed your design.
6. **The hard part.** What was the hardest import problem you encountered? Show how you handled it, or explain the remaining limitation. Include one failure case.
7. **Hive.** What did you find while using our product that we could do better? Be direct and specific.

Spend most of your time on points 2 through 6. The introduction and Hive feedback can be short.

If shortlisted, expect a short live discussion of your implementation and a small change to work through with us. Be ready to explain and adapt the code you submitted.